# COMP20003 Assignment 2: Stage 3 Report

Linked-List and Patricia-Trie Dictionary Complexity

Nhat Chi Nguyen (1492182)

## 1 Scope and theoretical expectations

The Stage 1 dictionary (`dict1`) stores all $n$ records in a singly linked list. Because duplicate keys are allowed and every matching record must be returned, this implementation scans the entire list for every query. The Stage 2 dictionary (`dict2`) stores keys in a binary Patricia trie. A search compares the query with the packed bit-stem in each visited node and follows one branch until it reaches a leaf or the first mismatching bit. Records with the same key are retained in input order at the same leaf.

Both programs report bit comparisons ($b$), node accesses ($n_c$), and whole-string comparisons ($s$). These counters measure different operations, so they should not be treated as interchangeable proxies for running time. Let $N$ be the number of records, $U$ the number of distinct keys, and $L$ the query length in bits, including the `\0` terminator. The expected lookup behaviour is:

| Structure | Counter | Theoretical search cost |
|---|---|---|
| Linked list | node/string | $\Theta(N)$ for this full-scan implementation |
| Linked list | bit | $O(NL)$ worst case; $\Theta(N)$ when $L$ is fixed |
| Patricia trie | bit | $\Theta(L)$ for a successful search; $O(L)$ for an unsuccessful one |
| Patricia trie | node | expected $O(\log U)$ for balanced branching; worst case $O(\min\{U, L\})$ |

**Table 1:** Predictions used to interpret the experiments.

The closest-candidate edit-distance work is excluded from the reported counters, as permitted by the specification. Consequently, the misspelling experiments measure only the cost of locating the mismatching subtree; enumerating and ranking its candidates is a separate, excluded cost. The report focuses on lookup rather than insertion.

## 2 Experimental design

A Python harness generated data, supplied queries to the submitted `dict1` and `dict2` binaries, and parsed each program's `b## n## s##` output. Comparison counts are deterministic for a fixed dataset and query, so all variability comes from which dataset and which queries were sampled, not from timing noise. To capture both sources, each reported point in Experiments 1 and 2 pools results from 5 independently generated datasets, each queried with 20 samples (100 samples per point in total), and reports the standard deviation across this pooled sample alongside the mean. Both programs were first checked against Ed's automated test suite and the provided reference outputs, so the experiments below measure comparison behaviour under implementations that passed the available tests and matched the provided reference outputs, rather than testing correctness itself.

**Experiment 1: growth and key density.** Ten sizes from $N = 50$ to $N = 40\,000$ were tested with 32-character lowercase keys. The *sparse* datasets used independently generated keys, which usually diverged within the first few bits. Each *dense* key differed from an all-`a` baseline at exactly two character positions, creating much longer pairwise agreement, with the position pair and character pair drawn from a precomputed bijection so that no two keys in a dataset could coincide. This experiment varies input size and the distribution of informative bits.

**Experiment 2: edit position.** At $N = 5\,000$, three query types were compared at both densities: an exact stored key, the same key with its last character changed (a *last-character edit*), and the same key with its first character changed (a *first-character edit*). The replacement character is drawn from the same alphabet as the stored keys, so an edit's position does not necessarily coincide with where the trie's search first diverges from the

query: the modified prefix may still follow a branch shared with a different stored key, especially under the dense generator's heavy prefix sharing.

All synthetic keys were unique, so $N = U$ in Experiments 1 and 2; this was asserted programmatically for every generated dataset rather than assumed. Modified queries were also checked to ensure they were not existing keys, redrawing the changed character when a collision was found.

**Experiment 3: provided data.** The provided `dataset_1353.csv` was queried using `test1353.in` (588 exact queries) and `testpart1353.in` (8 misspelt queries). Its variable-length, hierarchical keys and duplicate records provide a check beyond the two synthetic distributions.

# 3 Results

## 3.1 Growth with dataset size

For the linked list, node and string comparisons were exactly $N$ for every query in all 20 size–density combinations, confirming $\Theta(N)$ growth. Bit comparisons were also linear in $N$ for fixed-length keys, but density changed the constant substantially: Figure 1 shows the dense-to-sparse ratio growing from about $5\times$ at $N = 50$ to about $10\times$ at $N = 40\,000$, with wide sample-to-sample spread across datasets and queries at the dense end (visible from the error bars). The linked list stops comparing a stored key at its first mismatching bit, so long pairwise agreement makes each of its $N$ comparisons more expensive.

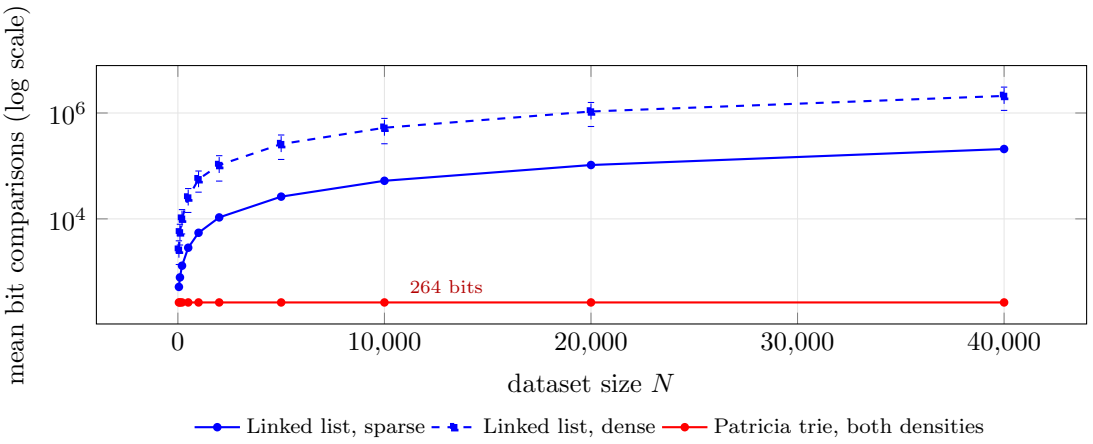



**Figure 1:** Mean bit comparisons per exact query, with error bars showing ±1 standard deviation across the pooled sample. Patricia results overlap at 264 bits with zero variance; linked-list results grow linearly, with a density-dependent constant and substantial dense-data spread.

For the Patricia trie, every exact query compared exactly 264 bits: 32 characters plus the 8-bit terminator, with zero variance across every dataset and query tested. This is independent of both $N$ and density because the stems on a root-to-leaf path partition the key bits, and it directly confirms the $\Theta(L)$ bound for a successful search (an unsuccessful search only bounds this above, by $O(L)$, since it may stop before all $L$ bits are consumed).

Node accesses depend instead on how the distinguishing bits partition the keys. On sparse data, the least-squares model $n_c = 1.015 \log_2 N + 1.385$ follows the observations closely (Figure 2), consistent with approximately balanced branching and expected $O(\log U)$ depth. Dense observations lie 2.6–4× above this sparse-data fit over the tested range, with substantial spread at every size. The fit is descriptive rather than a formal bound: path length remains bounded by $O(\min\{U, L\})$, and a fixed $L$ ultimately caps tree depth regardless of density. The larger observed depth for dense data is consistent with less even partitioning by the distinguishing bits, since Patricia compression already stores the shared prefix itself in a single stem; partition balance itself was not measured directly.

At $N = 40\,000$, relative to the linked list, the trie used 791× fewer bit comparisons and 2388× fewer node accesses on sparse data. On dense data the reductions were 7961× and 599×, respectively. These ratios are specific to this workload and should not be generalised as universal constants.

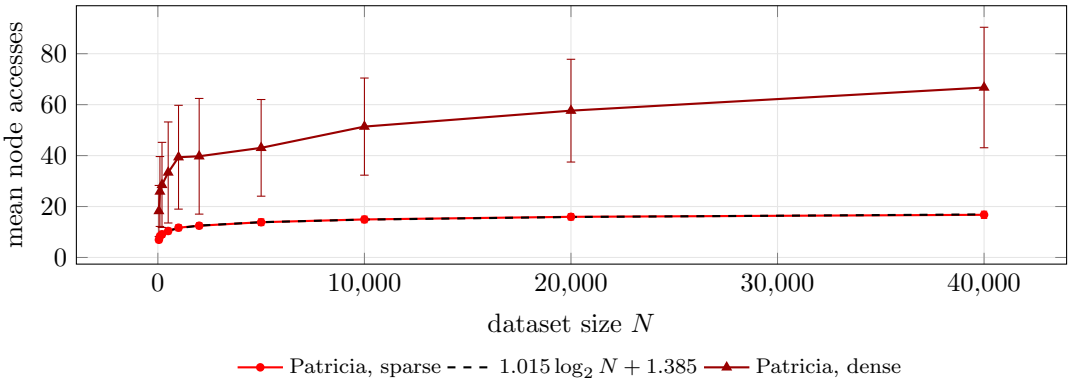



**Figure 2:** Patricia node accesses, with error bars showing ±1 standard deviation across the pooled sample. Sparse data track a logarithmic fit tightly; dense data have deeper paths with much greater spread over the measured range.

## 3.2 Edit position

Table 2 fixes $N = 5\,000$, pooling 5 datasets × 20 queries per row as in Experiment 1. Linked-list node and string counts remain exactly 5000 with zero variance, because the full scan is independent of query outcome. Patricia exact and last-character-edit queries have similar bit and node counts because almost the whole key is consumed before the decision. A first-character edit generally reduces bit comparisons, but it does not guarantee an immediate trie mismatch: the modified prefix may still follow a branch belonging to a different stored key, so the edit position and the trie's actual mismatch position can differ.

This distinction is visible directly in the data. On sparse data, a first-character edit reliably causes an early divergence (bit comparisons fall to $24.3 \pm 3.1$, a tight distribution), because sparse keys branch almost immediately and few other keys share the edited prefix. On dense data, the same edit is far less reliable: bit comparisons average $101.8 \pm 56.7$, both much higher than the sparse case and far more variable, because 94% of dense keys still begin with the unedited baseline character, so a first-character edit frequently lands on a prefix that other stored keys still share, and the search continues until a real divergence is found deeper in the tree. The node effect follows the same pattern: on sparse data a first-character edit changes the mean only from 13.7 to 12.7 nodes, well within one standard deviation, whereas on dense data it reduces the mean from 43.4 to 13.1 nodes, with a large standard deviation (18–20 nodes across all three dense query types) that reflects exactly this edit-position/mismatch-position gap. Bit and node comparisons therefore measure different things: bit comparisons track matched prefix length, largely independent of tree shape, while node comparisons track the compressed branching structure, which is far less stable for dense keys.

| Density | Query type | LL bit | LL node | PT bit | PT node |
|---|---|---|---|---|---|
| Sparse | Exact | $26195 \pm 655$ | 5000 | $264.0 \pm 0.0$ | $13.7 \pm 1.3$ |
| Sparse | Last-character edit | $26168 \pm 640$ | 5000 | $252.9 \pm 1.1$ | $14.0 \pm 1.1$ |
| Sparse | First-character edit | $25992 \pm 614$ | 5000 | $24.3 \pm 3.1$ | $12.7 \pm 1.4$ |
| Dense | Exact | $272285 \pm 119374$ | 5000 | $264.0 \pm 0.0$ | $43.4 \pm 17.9$ |
| Dense | Last-character edit | $236603 \pm 135990$ | 5000 | $253.8 \pm 0.8$ | $38.7 \pm 19.9$ |
| Dense | First-character edit | $30109 \pm 3661$ | 5000 | $101.8 \pm 56.7$ | $13.1 \pm 5.5$ |

**Table 2:** Mean ± standard deviation of comparisons per query at $N = 5\,000$, pooled across 5 datasets × 20 queries (100 samples) per row. LL = linked list; PT = Patricia trie.

The low counter values for a first-character edit require careful interpretation even when the edit does land near the tree's actual divergence point: a shallow mismatch can still leave a large descendant subtree, so candidate enumeration and edit-distance ranking may be expensive even though the counted search path is short. The table therefore reflects only prefix-lookup cost; total spell-correction time is a separate, unmeasured quantity.

### 3.3 Provided dataset and duplicate keys

The real dataset shows the same qualitative contrast (Table 3). The linked list again performs exactly 1353 node and string comparisons per query. The Patricia trie uses fewer than 20 node accesses on average, while its larger bit counts reflect the real keys' greater and variable lengths.

| Query file | LL bit | LL node | LL str. | PT bit | PT node | PT str. |
|---|---|---|---|---|---|---|
| `test1353.in` (588 exact) | 116490 | 1353 | 1353 | 525.9 | 17.4 | 1.0 |
| `testpart1353.in` (8 misspelt) | 37536 | 1353 | 1353 | 328.9 | 9.1 | 1.0 |

**Table 3:** Mean comparisons per query on `dataset_1353.csv` ($N = 1353$).

Duplicates affect the structures differently. In the linked list, each duplicate is another record node and therefore increases the full-scan cost. In the Patricia trie, records sharing a key are attached to one leaf, so multiplicity does not lengthen the search path; it only increases the work needed to output the matching record list. This separates $N$ from $U$: Patricia lookup cost depends primarily on the number of distinct keys, independent of how many records each key returns.

## 4 Discussion

The measurements support the predictions in Table 1. The linked list has $\Theta(N)$ node and string comparisons for every query, with prefix density changing only the constant in its bit count. Patricia bit comparisons are exactly bounded by key length for a successful search ($\Theta(L)$, confirmed with zero variance) and bounded above for an unsuccessful one ($O(L)$), while node depth is near logarithmic only when distinguishing bits split the key set reasonably evenly. The dense experiment therefore qualifies the average-case $O(\log U)$ claim while leaving Patricia's bit bound intact.

Whole-string comparisons illustrate why multiple counters are necessary: the linked list performs one for every record, whereas this Patricia implementation performs at most one after its bitwise descent. Bit and node counts expose different effects again in Table 2. Reporting only one counter would conceal either prefix-density cost or tree-shape cost.

With key length bounded, both structures use $\Theta(N)$ total space asymptotically: the linked list allocates $\Theta(N)$ record nodes, and a binary Patricia trie with $U \leq N$ leaves has at most $U - 1$ internal nodes, so its total storage is $O(N + UL)$ in the worst case and $O(N + U) = O(N)$ when $L$ is fixed. Patricia's practical disadvantage is therefore a constant-factor one, not a different asymptotic class: each internal node additionally stores a packed stem, a stem-length field, and two child pointers that the linked list does not need. This implementation was not instrumented for bytes allocated, so this is an asymptotic space observation rather than an empirical memory measurement.

Two limitations constrain interpretation. First, operation counts omit cache behaviour, allocation overhead, output, candidate enumeration, and edit distance; they should not be presented as wall-clock speedups. Second, each condition pools only five independently generated datasets; this captures both dataset-to-dataset and query-to-query spread (visible in Table 2 and the error bars of Figures 1–2) but is not a large sample, so the reported standard deviations themselves carry sampling uncertainty, particularly for the highly variable dense condition. The systematic variation in $N$, density, edit position, and real-data structure supports the growth conclusions, but the exact ratios should not be generalised.

## 5 Conclusion

The experiments confirm the linked list's $\Theta(N)$ full-scan cost and the Patricia trie's $\Theta(L)$ bitwise lookup for a successful search. Sparse keys produced approximately logarithmic Patricia node depth, while dense keys produced deeper, more variable paths, consistent with less even partitioning by their distinguishing bits. This difference affects finite-sample node cost while leaving the bit bound intact. Overall, the Patricia trie greatly reduces measured lookup work, especially as $N$ grows, while duplicate handling, candidate ranking, and constant-factor memory overhead determine the remaining trade-offs.